

HubStudio

Arquitetura do Projeto como cada parte foi feita

Gerenciador de redes sociais — agendamento de posts,
analytics e insights de IA.

React 18 · Vite · JavaScript · CSS puro
framer-motion · recharts · react-router-dom v6

Documento técnico de referência

1 Fundação

A base que sustenta tudo.

Stack: Vite + React 18 + JavaScript puro, com CSS puro e variáveis. Sem TypeScript, sem Tailwind.

Roteamento — rotas aninhadas

Uso `react-router-dom` v6. As rotas do dashboard ficam *dentro* de uma rota pai que renderiza o `DashboardLayout`. O layout tem a sidebar fixa + um `<Outlet/>` onde as páginas-filhas são injetadas — assim a sidebar nunca recarrega ao trocar de página.

Proteção de rotas

O `ProtectedRoute` tem 3 linhas: lê `isAuthenticated` do contexto e ou renderiza o filho, ou faz `<Navigate to="/entrar" replace/>`. Em React não há "middleware" — proteção de rota é renderização condicional.

Estado global via Context API

Três contextos (`Auth`, `Theme`, `Team`), cada um com seu hook (`useAuth()`, `useTheme()` ...). Não usei Redux porque o estado é pequeno; Context resolve.

DECISÃO DE ARQUITETURA — SERVIÇOS MOCK

Toda função em `src/services/` retorna `Promise.resolve(dados)`. Isso fixa o **contrato**: quando o backend real entrar, troca-se só o corpo da função (de mock para `fetch`) e nenhum componente muda.

2 Sistema de tema (dark mode)

O mecanismo mais reutilizado do projeto.

- O contexto guarda `theme`, inicializado lendo do `localStorage` (lazy initializer, pra não piscar).
- Um `useEffect` escreve `<html data-theme="dark">` no documento.
- No CSS, o seletor `[data-theme='dark']` **redefine as mesmas variáveis** (`--color-bg`, `--color-text` ...). Como todo componente usa `var(--color-bg)` e não cores fixas, o site inteiro troca de tema **sem um único re-render de cor no JS** — é puro CSS cascadeando.

DETALHE AVANÇADO

Ícones de redes pretas (TikTok/X) somem no fundo escuro. Por isso existe o helper `networkColor(id, theme)`: cada rede tem `color` e opcionalmente `darkColor`, e o helper devolve a versão certa pro tema.

3 Landing

Animação é a estrela.

A página toda usa **framer-motion**. Três padrões essenciais:

- **Animar ao rolar** — `whileInView` + `viewport={{ once: true }}` : o elemento começa invisível e anima quando entra na tela, uma vez só. É o efeito de seções "subindo".
- **Stagger (cascata)** — um container com `staggerChildren` faz os filhos animarem em sequência, não todos juntos.
- **Contador animado** — o número que sobe de 0 ao valor final. O segredo é o callback `onViewportEnter` : ao entrar na tela, dispara um `setInterval` que incrementa em 60 passos / 2s. Tem um guard pra não reiniciar ao rolar de novo.

SACADA DE PERFORMANCE

O HeroMockup e os FeaturePreviews **não são imagens** — são mini-interfaces falsas feitas em JSX + CSS (um dashboard de mentira, um gráfico fake). Dá leveza (sem PNG pesado) e fica nítido em qualquer tela.

4 Autenticação

- **Medidor de força de senha** — `calcStrength()` dá 1 ponto por critério (8+ caracteres, maiúscula+minúscula, número, símbolo) → score 0–4 → label "Frac/Média/Boa/Forte". Pontuação por regex, simples e legível.
- **Logos que trocam no dark mode** — cada tela escolhe via `theme === 'dark' ? logoHubDark : logoHub` .
- **Ícones OAuth** — SVGs inline com as cores oficiais de Google/Facebook, em vez de biblioteca, pra ter a cor exata da marca.

5 Dashboard Home — a parte mais complexa

Carregamento inteligente de dados + dashboard modular drag-and-drop.

5.1 — Carregamento de dados por dependência

Em vez de um `useEffect` gigante:

- Um `Promise.all` carrega de uma vez tudo que é estático (audiência, top posts, calendário...) — uma rajada paralela no mount.
- `useEffect` separados por filtro: um keyed em `[period, network]`, outro em `[granularity, network]`, outro em `[network]`. Ao trocar só o período, **só os stats refazem o fetch** — não recarrega o dashboard inteiro.

5.2 — Dashboard modular (peças de Lego)

Como o usuário arrasta os cards e a ordem persiste:

- **Registry de blocos** — cada card é uma entrada num objeto: `LEFT_BLOCKS = { kpis: <KpiGrid/>, ... }`. A ordem é só um **array de ids**. Separar "o que renderizar" de "em que ordem" é o que torna tudo reordenável.
- **Drag-and-drop com `Reorder` do `framer-motion`** — `<Reorder.Group values={order} onReorder={setOrder}>` + `<Reorder.Item value={id}>`. A lib cuida da física; eu só forneço array e setter. Sem nenhuma lib de drag-and-drop extra.
- `dragListener={editMode}` — o arraste só liga no modo "Personalizar". Fora dele, os cards são clicáveis normalmente. Um único prop liga/desliga toda a interação.
- **Persistência à prova de futuro** — `loadOrder()` reconcilia o array salvo no `localStorage` com o padrão: mantém a ordem do usuário, descarta ids inexistentes e adiciona blocos novos no fim. Um card novo numa atualização aparece sem quebrar o layout salvo.

5.3 — Gráfico de engajamento (recharts)

- **AreaChart com preenchimento em gradiente**: um `<linearGradient>` vai da cor da métrica (opacidade 0.28) até transparente — o degradê suave sob a linha.
- **O seletor de métrica acumula 3 funções**: é legenda, resumo (total via `useMemo`) e filtro (troca a série). Clicar muda a `dataKey` e a cor de tudo.
- **Cores via CSS variables dentro do SVG** (`stroke="var(--color-border)"`) — o gráfico respeita o dark mode automaticamente.

6 Composer (criar / editar post)

O mais complexo em gerência de estado.

O desafio: o mesmo post pode ir pra várias redes, **cada uma com conteúdo diferente**.

- **Estado por rede** — o coração é `contentByNetwork: { instagram: {title, content, media}, ... }`. Um `activeNetwork` define qual rede se edita; formulário e preview leem desse mapa.
- **PhonePreview** — um celular desenhado em CSS com Dynamic Island (a pílula do iPhone) e preview específico por rede, atualizado em tempo real.
- **3 features de IA** — insight de formato que mais engaja, geração de legenda e hashtags, e melhores horários. Funções mock que seguem o contrato de `Promise`, prontas pra virar IA real.
- **DateTimePicker custom** — calendário do zero (grid de 42 células = 6 semanas). O detalhe esperto: recomendações de horário cientes do tempo atual — se a data é hoje, filtra horários que já passaram. Nunca sugere 14h se já são 15h.

BUG CLÁSSICO RESOLVIDO

A barra de ações `position: fixed` "teleportava". Causa: a animação global tinha `transform`, e **qualquer ancestral com `transform` vira o containing block de um filho `fixed`**. Solução: uma animação só de opacidade, sem transform.

7 Configurações & Suporte

Composição + deep-linking.

Configurações tem 7 abas, mas o arquivo principal é só composição: cada aba é um componente. O truque é o **deep-linking com `useSearchParams`**: a aba ativa vem da URL (`?tab=redes`), então dá pra linkar direto pra uma aba (a sidebar leva ao perfil, o tour leva às redes).

Suporte segue a mesma filosofia: montagem de blocos pequenos + um chat flutuante mockado. Esse padrão de **"página = composição de sub-componentes"** se repete no projeto todo — evita arquivos de 1500 linhas.

8 Hook de atalhos de teclado

Um hook que registra um listener global de `keydown`. Dois detalhes maduros:

- **Normaliza `mod`**: `e.metaKey || e.ctrlKey` — o mesmo atalho funciona como Cmd no Mac e Ctrl no Windows.
- **Guard de digitação**: atalhos de tecla única (`n`, `?`) só disparam se você não estiver num input — senão digitar "n" abriria "novo post" no meio de uma frase. Combos com modificador (`Ctrl+K`) sempre funcionam.



Trunfos para a defesa

Pergunta provável	Resposta-chave
Como o dark mode funciona sem lib?	<code>data-theme</code> no <code><html></code> + variáveis CSS redefinidas; zero re-render de cor no JS
Como o dashboard é modular?	Registry (id→JSX) + array de ordem + <code>Reorder</code> do framer-motion + <code>loadOrder</code> reconciliando o localStorage
Como integra o backend?	Padrão de serviços mock com <code>Promise</code> — troca só o corpo da função
Como protege rotas?	<code>ProtectedRoute</code> com renderização condicional + <code>Navigate</code>
Otimização de performance?	<code>useEffect</code> separados por filtro + <code>useMemo</code> nos cálculos de gráfico
Bug interessante que resolveu?	<code>position: fixed</code> + ancestral com <code>transform</code> (containing block)